

# Тема: Многопоточное программирование

## Реализовать консольное приложение, удовлетворяющее следующим требованиям:

Общие технические требования к реализации

Каждое задание должно быть выполнено с соблюдением следующих правил:

### 1. Управление потоками и жизненный цикл

- **Запрет на Busy Waiting (Активное ожидание):** Потоки-обработчики не должны потреблять ресурсы CPU в пустых циклах `while(true)`. Если задач нет, поток обязан переходить в состояние ожидания (`WAITING`) с помощью `wait()`, `Condition.await()` или блокирующих очередей.
- **Корректное завершение:** Программа должна завершаться не по `System.exit()`, а через механизм прерывания (`interrupt()`) или использование `volatile boolean` флага. Все потоки должны корректно выйти из метода `run()`.
- **Именование:** Каждому потоку должно быть присвоено осмысленное имя (например, `Thread(runnable, "Manager-Premium")`) для удобства отладки и чтения логов.

### 2. Синхронизация и работа с данными

- **Атомарность операций:** Все действия с общими ресурсами (добавление в очередь, извлечение, изменение счетчиков статистики) должны быть атомарными. Не допускайте ситуаций `Race Condition` при проверке `size()` или `isEmpty()` перед извлечением данных.
- **Минимизация области блокировки:** Код внутри блоков `synchronized` или под защитой `Lock` должен содержать только операции с общими данными. **Запрещено** вызывать `Thread.sleep()` внутри синхронизированного блока/метода.
- **Thread-Safety коллекций:** Используйте либо стандартные коллекции (`LinkedList`, `ArrayList`) с явной синхронизацией по объекту-монитору, либо специализированные классы из пакета `java.util.concurrent`.

### 3. Сбор статистики и логирование

- **Потокобезопасная статистика:** Для счетчиков используйте `AtomicInteger/AtomicLong` или синхронизируйте доступ к обычным переменным.
- **Информативность вывода:** Каждое значимое событие (поступление клиента, начало обслуживания, завершение работы) должно выводиться в консоль с указанием имени текущего потока: `System.out.println(Thread.currentThread().getName() + ": сообщение")`.
- **Финальный отчет:** После остановки симуляции программа должна вывести итоговую статистику: общее число обработанных объектов и максимальные зафиксированные длины очередей.

### 4. Обработка исключений

- **InterruptedException:** При поимке исключения поток должен либо завершить работу, либо восстановить статус прерывания через `Thread.currentThread().interrupt()`.

### 1. Таксопарк: Битва за VIP-заказы

**Сюжет:** В городе работает единая диспетчерская служба и **два независимых водителя** (потоки-обработчики). Заказы поступают двух типов: «VIP» и «Эконом».

#### Требования к логике:

- **Генераторы:** Два потока создают заказы. VIP-заказы поступают редко (раз в 500–1000 мс), Эконом-заказы — часто (раз в 200–400 мс).
- **Очереди:** Две независимые очереди.
- **Обработчики:** Два потока-водителя. Каждый, освободившись, проверяет сначала VIP-очередь. Если она пуста — берет заказ из Эконома.
- **Параметры:** Время поездки (обработки) — 600 мс.
- **Цель:** Вывести, сколько заказов «Эконом» успели выполнить водители за 30 секунд, и не «простаивал» ли VIP-клиент, пока оба водителя были заняты эконом-заказами.

**Рекомендация:** Использовать `ReentrantLock` для защиты очередей от одновременного захвата одного заказа двумя водителями.

### 2. Складской терминал: Режим аврала (3 погрузчика)

**Сюжет:** На терминал прибывают фуры с товаром и мелкие газели. На складе работают **три погрузчика** (3 потока-обработчика).

#### Требования к логике:

- **Генераторы:** Один поток создает фуры, другой — газели (интервал 400–800 мс).
- **Очереди:** Очередь «Зона разгрузки фур» и «Зона погрузки газелей».
- **Обработчики:** Три погрузчика. Если в очереди более 5 фур, **все три** погрузчика обязаны работать только с ними (приоритет). Если фур меньше, свободные погрузчики могут помогать газелям.
- **Статистика:** Работа 40 секунд.
- **Цель:** Зафиксировать пиковую длину очереди газелей в моменты «наплыва» фур.

**Рекомендация:** Использовать `notifyAll()`, чтобы при освобождении погрузчика пробуждались все спящие потоки для проверки приоритетных условий.

### 3. Поликлиника: Прием у дежурных врачей (N ядер)

**Сюжет:** В период эпидемии количество врачей (потоков) соответствует числу ядер процессора.

#### Требования к логике:

- **Генераторы:** Поток «Пациенты по записи» и поток «Экстренные случаи».
- **Обработчики:** Количество потоков = `Runtime.getRuntime().availableProcessors()`.
- **Логика:** Врачи разбирают пациентов. Если появляется «экстренный», любой освободившийся врач обязан взять его первым.
- **Специфика:** Реализовать общий счетчик «Исцеленных» через `AtomicInteger` или `synchronized`.
- **Цель:** Сравнить общую пропускную способность системы при разном количестве выделенных потоков-врачей (1, 2, 4).

### 4. Издательство: Печать газет и листовок

**Сюжет:** В типографии стоят две печатные машины. Первая — современная (универсальная), вторая — старая (печатает только листовки).

#### Требования к логике:

- **Генераторы:** Поток «Заказы на газеты» и поток «Заказы на листовки».
- **Обработчики:**
  - о Поток 1 (Машина №1): Проверяет очередь газет, если пусто — берет листовки.
  - о Поток 2 (Машина №2): Берет **только** листовки.
- **Конфликт:** Если в очереди только листовки, обе машины могут пытаться взять заказ одновременно.
- **Цель:** Определить максимальное время ожидания для заказа на газету, если машина №1 занята листовками.

### 5. Аэропорт: Гейт на посадку (Бизнес-приоритет)

**Сюжет:** Идет посадка на рейс. Работают **два инспектора**. Перед ними две очереди.

#### Требования к логике:

- **Генераторы:** Пассажиры бизнес-класса и эконом-класса (интервал 100–300 мс).
- **Обработчики:** Два инспектора проверяют документы (150 мс на человека).
- **Логика:** Пока в очереди бизнес-класса есть люди, **оба** инспектора работают только с ними. Эконом-класс начинает двигаться только при полностью пустой бизнес-очереди.
- **Цель:** Вывести среднее и максимальное количество людей, скопившихся в эконом-классе к моменту завершения посадки.

6. СТО: Двухэтапный ремонт (Цепочка потоков)

**Сюжет:** Автосервис, где машина сначала проходит «Диагностику», а затем «Ремонт».

**Требования к логике:**

- **Обработчики:** 2 потока-диагноста и 1 поток-механик.
- **Очереди:**
  1. Общая очередь на въезд.
  2. Промежуточная очередь «После диагностики».
- **Логика:** Диагносты работают быстро (300 мс), механик — медленно (800 мс). Если промежуточная очередь заполнена (5 машин), диагносты должны прекратить работу и ждать.
- **Цель:** Найти максимальный размер очереди на въезд в СТО.

7. Ресторан: Повара и Официанты (Ограниченное окно)

**Сюжет:** Кухня выдает блюда в «окно выдачи», откуда их забирают официанты.

**Требования к логике:**

- **Обработчики:** 2 повара (генераторы) и 3 официанта (потребители).
- **Ресурс:** Окно выдачи вмещает только 5 тарелок.
- **Логика:** Повара останавливаются, если окно полно. Официанты «засыпают», если окно пусто.
- **Цель:** Вывести статистику «простоя» поваров (сколько раз им приходилось ждать официантов).

8. Порт: Разгрузка тяжелых судов (Захват ресурсов)

**Сюжет:** В порту 4 крана. Прибывают малые суда (нужен 1 кран) и сверхтяжелые танкеры (нужно сразу 2 крана).

**Требования к логике:**

- **Генераторы:** Потоки малых и тяжелых судов.
- **Обработчики:** 4 потока-крана.
- **Специфика:** Тяжелое судно должно «заблокировать» два свободных крана. Если свободен только один, оно ждет, не давая малым судам занять этот кран (предотвращение голодания).
- **Цель:** Вывести максимальную очередь тяжелых судов.

9. Банк: Перерыв на инкассацию

**Сюжет:** В отделении 2 окна обслуживания. Иногда приходит инкассатор (приоритетный поток).

**Требования к логике:**

- **Генераторы:** Поток клиентов. Раз в 10 секунд появляется инкассатор.
- **Обработчики:** 2 окна.
- **Логика:** Когда появляется инкассатор, **первое освободившееся** окно обязано взять его. Если оба окна заняты, инкассатор ждет «в голове» всех очередей.
- **Цель:** Рассчитать, как сильно приход инкассатора увеличивает среднюю длину очереди обычных клиентов.

10. Супермаркет: Экспресс-касса и «Забывчивый покупатель»

**Сюжет:** В магазине работают **три кассы** (3 потока-обработчика). Первая касса — «Экспресс» (только до 5 товаров), остальные две — универсальные.

**Требования к логике:**

- **Генераторы:** Поток «Покупатели с корзинами» (1–5 товаров) и поток «Покупатели с тележками» (10–30 товаров).
- **Очереди:** Две очереди: «Экспресс-линия» и «Основная очередь».
- **Обработчики:**
  - о Касса №1: Берет людей из экспресс-линии. Если там пусто, берет одного человека из основной очереди.
  - о Кассы №2 и №3: Берут людей из основной очереди. Если она пуста, помогают экспресс-линии.
- **Нюанс:** Время обслуживания зависит от количества товаров (1 товар = 50 мс).
- **Цель:** Вывести максимальную длину основной очереди и среднее время ожидания для «тележек».

#### 11. Горнолыжный курорт: Подъемник и VIP-скипасс

**Сюжет:** На вершину горы ведет один подъемник, вмещающий **4 человека одновременно** (групповой обработчик).

##### Требования к логике:

- **Генераторы:** Группа «Туристы» и группа «Инструкторы» (VIP).
- **Очереди:** Две очереди у турникета.
- **Логика:** Как только кабина подъезжает, система сначала заполняет места инструкторами. Если инструкторов меньше 4, оставшиеся места занимают обычные туристы.
- **Параметры:** Кабина уходит каждые 2 секунды.
- **Цель:** Посчитать, сколько циклов подъемника пришлось прождать обычному туристу в пиковые моменты.

#### 12. СТО: Покрасочный цех с сушкой

**Сюжет:** Машина проходит два этапа у разных мастеров: «Покраска» (2 потока) и «Сушка» (1 поток).

##### Требования к логике:

- **Очереди:** Очередь на покраску и промежуточная камера ожидания перед сушкой.
- **Ограничение:** Камера сушки вмещает только одну машину. Покрасочный цех должен остановиться, если в камере ожидания уже накопилось 3 машины (чтобы не испортить краску ожиданием).
- **Параметры:** Покраска длится 1000 мс, сушка — 1500 мс.
- **Цель:** Вывести время простоя покрасочных потоков из-за «затора» на сушке.

#### 13. Автоматическая сортировка почты

**Сюжет:** Роботизированный склад с **двумя манипуляторами** (2 потока). Поступают посылки «Авиа» (срочные) и «Наземные».

##### Требования к логике:

- **Обработчики:** Два робота. Робот №1 — настроен только на «Авиа». Робот №2 — универсальный.
- **Логика:** Если Робот №2 видит «Авиа-посылку», он обязан бросить наземную сортировку и помочь Роботу №1 (приоритет скорости).
- **Сбор статистики:** Работа 30 секунд.
- **Цель:** Вывести максимальное количество наземных посылок, скопившихся в ожидании, пока оба робота занимались срочной почтой.

#### 14. Пожарная станция: Диспетчерская 112

**Сюжет:** На пульт поступают вызовы: «Пожар» (высокий приоритет) и «Котенок на дереве» (низкий). Работают **три оператора**.

##### Требования к логике:

- **Обработчики:** 3 потока-оператора.
- **Логика:** Если поступает сигнал «Пожар», а все операторы заняты «котятками», один из операторов должен максимально быстро завершить текущий разговор (имитация через прерывание потока или флаг) и взять критический вызов.
- **Цель:** Замерить время задержки обработки критического вызова.

#### 15. Парковка бизнес-центра: Арендаторы и Гости

**Сюжет:** На парковке всего 10 мест. Есть два заезда.

##### Требования к логике:

- **Генераторы:** Машины сотрудников (арендаторов) и машины гостей.
- **Очереди:** Две линии перед шлагбаумом.
- **Логика:** Если на парковке осталось последнее место, оно резервируется только для сотрудника. Гость может заехать только если свободных мест 2 и более.
- **Цель:** Вывести максимальное количество гостей, развернутых системой (или ждущих в очереди) за 1 минуту.

#### 16. Облачное хранилище: Загрузка и Чтение

**Сюжет:** Сервер обрабатывает запросы на чтение данных (Read) и запись (Write). Имитация работы с диском (N ядер).

##### Требования к логике:

- **Обработчики:** Количество потоков равно числу логических ядер CPU.
- **Логика:** Чтение выполняется быстро (20 мс), запись медленно (100 мс). Запись имеет приоритет: если пришел запрос на запись, новые запросы на чтение не берутся в работу, пока запись не завершится.
- **Цель:** Показать падение скорости чтения (latency) при увеличении интенсивности записи.

### 17. Интеллектуальный светофор

**Сюжет:** Перекресток главной и второстепенной дорог. Светофор (обработчик) переключает фазы.

**Требования к логике:**

- **Генераторы:** Потоки машин с двух направлений.
- **Логика:** Светофор горит зеленым для главной дороги по умолчанию. Переключается на второстепенную только если там накопилось более 5 машин ИЛИ если на главной дороге пусто.
- **Цель:** Вывести максимальную пробку на второстепенной дороге.

### 18. Биржа: Брокер и типы ордеров

**Сюжет:** Брокерский терминал обрабатывает заявки на покупку (Buy) и продажу (Sell).

**Требования к логике:**

- **Генераторы:** Поток рыночных заявок (выполнить немедленно) и лимитных (могут ждать).
- **Обработчик:** 1 поток-ядро биржи.
- **Логика:** Рыночные заявки всегда в голове очереди. Лимитные обрабатываются только если рынок «спокоен» (нет входящих рыночных заявок в течение 200 мс).
- **Цель:** Определить, сколько лимитных заявок «протухнет» (не будет обработано) за сессию.

### 19. Железнодорожный переезд: Сапсан и Электрички

**Сюжет:** Однопутный участок пути, через который проходят поезда двух типов: «Скоростной Сапсан» и «Пригородная электричка».

**Требования к логике:**

- **Генераторы:** Поток «Сапсанов» (редко, раз в 5000 мс) и поток «Электричек» (часто, раз в 1000 мс).
- **Обработчик:** Один диспетчер (контроллер пути). Время прохождения участка — 1500 мс.
- **Логика:** Сапсан имеет абсолютный приоритет. Если диспетчер видит в очереди Сапсан, он не выпускает на путь ни одну электричку, пока Сапсан не проедет.
- **Нюанс:** Если электричка уже выехала на путь, Сапсан должен ждать на въезде.
- **Цель:** Вывести максимальную пробку из электричек, возникшую из-за приоритета скоростных поездов.

### 20. Сборочный цех: Корпуса и Электроника

**Сюжет:** Для сборки одного робота (финальный продукт) нужны две детали: Корпус и Плата. Работают два мастера (2 потока-сборщика).

**Требования к логике:**

- **Генераторы:** Поток «Изготовитель корпусов» и поток «Изготовитель плат». Скорость разная.
- **Очереди:** Две корзины (склада) — для корпусов и для плат. Вместимость каждой — 10 штук.
- **Обработчики:** Два мастера. Чтобы собрать робота, мастер должен взять одну деталь из каждой корзины.
- **Специфика:** Если в одной из корзин пусто, мастер «засыпает» прямо у конвейера.
- **Цель:** Вывести среднее количество готовых роботов за 30 секунд и остаток деталей в каждой корзине.

### 21. Центр обработки данных: Мажоритарный сервер

**Сюжет:** В системе работают три вычислительных ядра (3 потока). Поступают задачи двух уровней сложности.

**Требования к логике:**

- **Генераторы:** «Простые задачи» (выполняются на 1 ядре) и «Сложные задачи» (требуют одновременной работы всех 3 ядер для параллельных вычислений).
  - **Логика:** Если поступает «Сложная задача», свободные ядра не берут «Простые», а ждут освобождения остальных ядер, чтобы начать совместную работу.
  - **Цель:** Зафиксировать время простоя ядер в ожидании объединения для сложной задачи.
- Рекомендация:** Использовать `CyclicBarrier` или `CountDownLatch`.

### 22. Нефтебаза: Заправка танкеров

**Сюжет:** На нефтебазе есть одна основная труба (общий ресурс) и два причала.

**Требования к логике:**

- **Генераторы:** Танкеры компании-владельца (приоритет) и частные танкеры.
- **Обработчики:** Два потока-причала. Но труба подачи нефти — одна. Одновременно качать нефть может только один танкер (даже если причалов два).
- **Логика:** Приоритетные танкеры всегда первые в очереди к трубе.
- **Цель:** Определить эффективность использования причалов: сколько времени один причал простаивал, пока другой качал нефть.

23. Коворкинг: Очередь к единственному принтеру

**Сюжет:** В офисе работают 5 сотрудников. У них один принтер.

**Требования к логике:**

- **Генераторы:** 5 потоков-сотрудников, которые генерируют документы (задачи на печать) случайного объема (количество страниц).
- **Обработчик:** 1 поток-принтер. Печать одной страницы — 100 мс.
- **Логика:** Принтер печатает документы по порядку, но если документ очень большой (более 10 страниц), он «прерывается» после каждых 5 страниц, чтобы пропустить короткий документ (1–2 страницы) из очереди, если такой появился.
- **Цель:** Вывести время ожидания для владельцев «коротких» документов.

24. Склад: Инвентаризация vs Работа

**Сюжет:** На складе работают грузчики, но раз в 10 секунд приходит Ревизор.

**Требования к логике:**

- **Генераторы:** Поток клиентов, забирающих товар.
- **Обработчики:** 2 потока-грузчика.
- **Логика:** Когда приходит Ревизор (высокоприоритетный поток), грузчики должны закончить текущую выдачу и заблокировать склад для выдачи новых товаров, пока Ревизор не «пересчитает» остатки (имитация 2000 мс).
- **Цель:** Вывести пиковую очередь клиентов, скопившуюся во время инвентаризации.

25. Фитнес-клуб: Очередь в душ

**Сюжет:** В раздевалке всего две душевые кабинки.

**Требования к логике:**

- **Генераторы:** Клиенты после «Групповой тренировки» (приходят толпой в 10 человек раз в 15 сек) и «Одиночные посетители» (заходят по одному раз в 3 сек).
- **Обработчики:** 2 потока-кабинки. Время в душе — от 500 до 1500 мс.
- **Логика:** Одиночные посетители имеют приоритет, чтобы не стоять в очереди за «толпой».
- **Цель:** Посчитать максимальное время ожидания для участников групповой тренировки.

26. Редакция: Корректор и Журналисты

**Сюжет:** 3 журналиста пишут статьи. Один корректор их проверяет.

**Требования к логике:**

- **Генераторы:** 3 потока-журналиста. Статья пишется 2–4 сек.
- **Очередь:** Корзина «На проверку» вмещает только 2 статьи. Если корзина полна, журналисты перестают писать и ждут.
- **Обработчик:** 1 корректор. Проверяет статью 1 сек.
- **Цель:** Посчитать суммарное время «творческого простоя» журналистов.

27. Горная дорога: Реверсивное движение

**Сюжет:** Узкий мост, на котором может находиться только **одна машина**. Потоки машин идут с севера и с юга.

**Требования к логике:**

- **Генераторы:** Машины «Север» и машины «Юг».
- **Обработчик:** Мост (общий ресурс). Время проезда — 500 мс.
- **Логика:** Если по мосту едет машина с севера, следующая машина с юга должна ждать. Чтобы не было «голодания», если с одной стороны скопилось более 3 машин, сторона обязана уступить дорогу встречному потоку.
- **Цель:** Вывести максимальную длину затора с каждой стороны.

28. Кухня: Фритюр и Повара

**Сюжет:** В ресторане 2 повара и **одна фритюрница**, в которой можно готовить одновременно 2 порции картошки.

**Требования к логике:**

- **Генераторы:** Заказы на «Картофель-фри» и «Бургеры» (не требуют фритюра).
- **Обработчики:** 2 потока-повара.
- **Логика:** Повар может делать бургер сам по себе, но для картофеля ему нужно занять «слот» во фритюрнице.
- **Специфика:** Фритюрница — это Semaphore (2). Заказы на фритюр имеют более высокий приоритет (т.к. остывают быстрее).
- **Цель:** Вывести статистику использования фритюрницы (среднюю загрузженность слотов).